

Task 1: Data Cleaning & Preprocessing - UNSW-NB15

This notebook performs: missing value imputation, one-hot encoding, ordinal encoding, outlier capping (winsorizing), and feature scaling.

All steps are applied to the training set and then transformed to the test set without data leakage.

Tools used: Python, Pandas, NumPy, Matplotlib/Seaborn, Scikit-learn

Cell 1: Import all required libraries: pandas, numpy, matplotlib, seaborn, and scikit-learn modules.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import OrdinalEncoder, StandardScaler
from sklearn.impute import SimpleImputer

%matplotlib inline
```

Cell 2: Load the training set, test set, and optional features description file.

```
train_df = pd.read_csv('UNSW-NB15/UNSW_NB15_training-set.csv')
test_df = pd.read_csv('UNSW-NB15/UNSW_NB15_testing-set.csv')

try:
    features_desc = pd.read_csv('UNSW-NB15/UNSW-NB15_features.csv',
                                encoding='cp1252')
    print("Features file loaded with cp1252 encoding")
except:
    print("Could not load features file – continuing without it")

print("Training set shape:", train_df.shape)
print("Testing set shape:", test_df.shape)

Features file loaded with cp1252 encoding
Training set shape: (82332, 45)
Testing set shape: (175341, 45)
```

Cell 3: Explore the data: column data types and missing values.

```
print("\nColumn data types:")
print(train_df.dtypes.value_counts())
```

```

missing = train_df.isnull().sum()
missing_pct = 100 * missing / len(train_df)
missing_table = pd.DataFrame({'Missing': missing, 'Percentage':
missing_pct})
print("\nMissing values:\n", missing_table[missing_table['Missing'] >
0])

```

```

Column data types:
int64      30
float64    11
str         4
Name: count, dtype: int64

```

```

Missing values:
Empty DataFrame
Columns: [Missing, Percentage]
Index: []

```

Cell 4: Handle missing values: median for numerical, most frequent for categorical.

```

num_cols = train_df.select_dtypes(include=['int64',
'float64']).columns
cat_cols = train_df.select_dtypes(include=['object',
'string']).columns

imputer_num = SimpleImputer(strategy='median')
train_df[num_cols] = imputer_num.fit_transform(train_df[num_cols])
test_df[num_cols] = imputer_num.transform(test_df[num_cols])

imputer_cat = SimpleImputer(strategy='most_frequent')
train_df[cat_cols] = imputer_cat.fit_transform(train_df[cat_cols])
test_df[cat_cols] = imputer_cat.transform(test_df[cat_cols])

print("Missing values after imputation:",
train_df.isnull().sum().sum())

```

```

Missing values after imputation: 0

```

Cell 5: Encode categorical features: one-hot for `proto` and `service`, ordinal for `state`.

```

train_df = pd.get_dummies(train_df, columns=['proto', 'service'],
drop_first=True)
test_df = pd.get_dummies(test_df, columns=['proto', 'service'],
drop_first=True)
test_df = test_df.reindex(columns=train_df.columns, fill_value=0)

all_states = pd.concat([train_df['state'], test_df['state']]).unique()
enc = OrdinalEncoder(categories=[all_states],

```

```

handle_unknown='use_encoded_value', unknown_value=-1)
train_df['state_encoded'] = enc.fit_transform(train_df[['state']])
test_df['state_encoded'] = enc.transform(test_df[['state']])
train_df.drop('state', axis=1, inplace=True)
test_df.drop('state', axis=1, inplace=True)

print("Shape after encoding:", train_df.shape)

Shape after encoding: (82332, 185)

```

Cell 6: Outlier capping (winsorizing) using NumPy for IQR calculation and vectorized clipping.

We use NumPy's `percentile` and `clip` functions for efficiency and clarity.

```

num_features = ['dur', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl',
                'sload', 'dload', 'sinpkt', 'dinpkt', 'sjit', 'djit']

def cap_outliers_iqr_numpy(df, column):
    data = df[column].values # extract as NumPy array
    Q1 = np.percentile(data, 25)
    Q3 = np.percentile(data, 75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df[column] = np.clip(data, lower, upper)
    return df

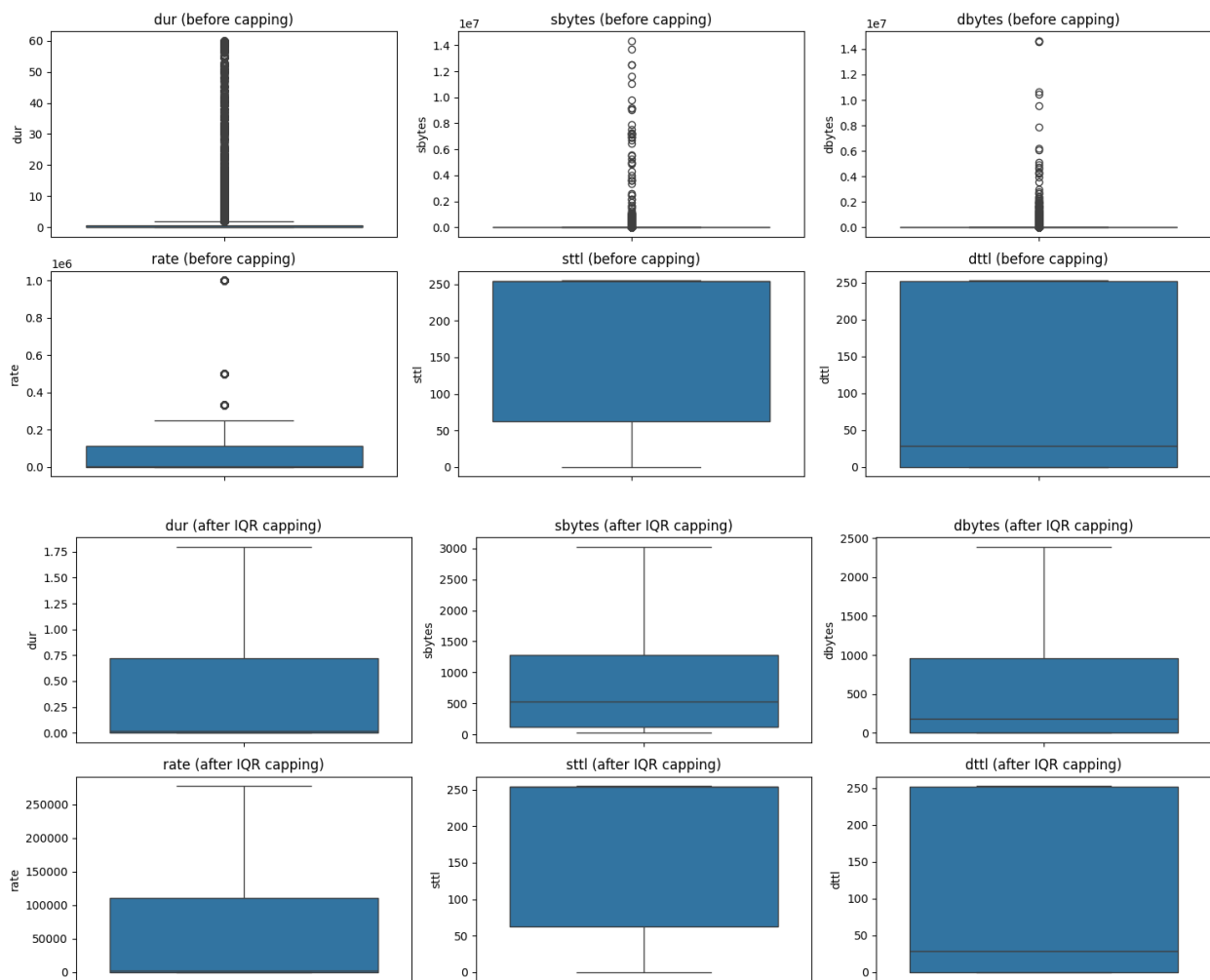
# Visualize before capping
plt.figure(figsize=(15, 6))
for i, col in enumerate(num_features[:6]):
    plt.subplot(2, 3, i+1)
    sns.boxplot(y=train_df[col])
    plt.title(f'{col} (before capping)')
plt.tight_layout()
plt.show()

# Apply capping to training set only (test remains untouched)
for col in num_features:
    train_df = cap_outliers_iqr_numpy(train_df, col)

# Visualize after capping
plt.figure(figsize=(15, 6))
for i, col in enumerate(num_features[:6]):
    plt.subplot(2, 3, i+1)
    sns.boxplot(y=train_df[col])
    plt.title(f'{col} (after IQR capping)')
plt.tight_layout()
plt.show()

```

```
print("Outliers capped (no rows removed). Training shape unchanged:",
      train_df.shape)
```



```
Outliers capped (no rows removed). Training shape unchanged: (82332,
185)
```

Cell 7: Standardize numerical features using `StandardScaler`. Afterwards, verify using NumPy that mean ≈ 0 and std ≈ 1 .

```
scaler = StandardScaler()
train_df[num_features] = scaler.fit_transform(train_df[num_features])
test_df[num_features] = scaler.transform(test_df[num_features])

# NumPy-based verification
means = train_df[num_features].mean().values
stds = train_df[num_features].std().values

print("Mean after scaling (close to 0):", np.round(means, 6))
```

```

print("Std after scaling (close to 1):", np.round(stds, 6))
print("All means ~0?", np.allclose(means, 0, atol=1e-6))
print("All stds ~1?", np.allclose(stds, 1, atol=1e-6))

Mean after scaling (close to 0): [ 0. -0.  0.  0. -0.  0. -0. -0.  0.
 -0. -0.  0.]
Std after scaling (close to 1): [1.0000006 1.0000006 1.0000006 1.0000006
 1.0000006 1.0000006 1.0000006 1.0000006 1.0000006
 1.0000006 1.0000006 1.0000006 1.0000006]
All means ~0? True
All stds ~1? True

```

Cell 8: Final check and save cleaned datasets.

```

print("Final training set shape:", train_df.shape)
print("Final test set shape:", test_df.shape)

train_df.to_csv('UNSW_NB15_training_cleaned.csv', index=False)
test_df.to_csv('UNSW_NB15_testing_cleaned.csv', index=False)
print("Cleaned files saved successfully.")

Final training set shape: (82332, 185)
Final test set shape: (175341, 185)
Cleaned files saved successfully.

```

Summary

1. **Encoding:** One-hot for `proto` and `service`; ordinal for `state` with unseen handling.
2. **Outliers:** Capped (winsorized) using NumPy's `IQR` and `clip` – preserves attack traffic.
3. **Test set integrity:** No outlier capping on test; only transformation from train.
4. **NumPy usage:** Explicitly used for `IQR`, clipping, and post-scaling verification.
5. **Tools:** Python, Pandas, NumPy, Matplotlib/Seaborn, Scikit-learn all utilized.